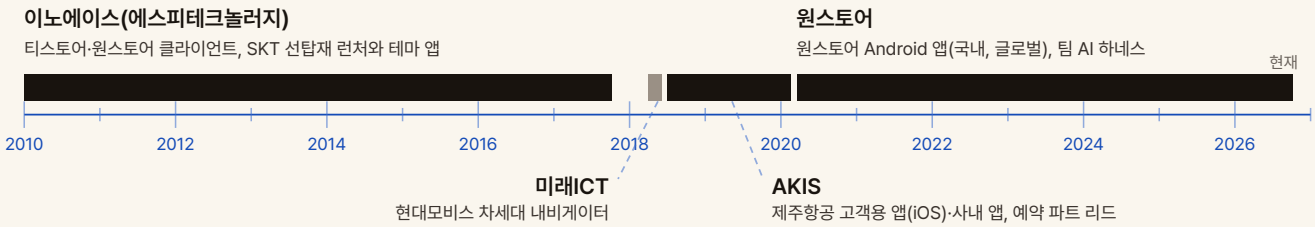
# 황보민

원스토어 클라이언트 팀(9명) 기술 리드 · Android 개발자 · 팀 AI 하네스 구축

010-4354-1115
1stpie@naver.com
경기 안양시

## 팀이 AI와 일하는 방식을 만들어 키우고, 서비스 중인 앱의 구조를 바꿔 왔습니다.

원스토어 클라이언트 팀(9명)의 기술 리드입니다. 2026년 1월부터 팀의 AI 개발 하네스를 설계해 단계적으로 키워 왔고, 지금은 팀원 전원이 이 하네스로 개발합니다. 반복 업무를 스킬로 만들어 자동화하고, 훅과 권한으로 에이전트를 제약하며, 업무 도메인별 에이전트와 개발 루프를 설계했습니다. 그 전에는 서비스 중인 국내·글로벌 원스토어 앱의 아키텍처 전환을 이끌었고, 기술 리드로서 팀의 코드 리뷰 문화와 아키텍처 표준을 세웠습니다. AKIS에서는 제주항공 iOS(Swift) 앱을 담당하고, 항공 예약 파트를 리드하며 웹 개발도 함께 했습니다.




| | |
|---|---|
| 팀 AX 개발·운영 환경 구축 | 룰과 에이전트 매뉴얼, 업무의 스킬화, 훅·권한 기반 제약, 도메인별 에이전트와 개발 루프, 로컬 LLM Wiki로 하네스를 만들어 팀 전원이 씁니다. |
| 기술 리드 | 팀의 PR·코드 리뷰 문화와 아키텍처 표준을 세워 합의를 이끌었고, 기반 기술 도입을 결정하고 기술 전환 과제를 주도했습니다. |
| 앱 아키텍처·기반 기술 전환 | 서비스 중인 국내·글로벌 앱에 5계층 Clean Architecture와 모듈 구조를 도입하고, 레거시 구조를 MVVM·MVI로, 비동기·DI를 Coroutine·Flow·Hilt로 바꿨습니다. |
| 모바일 앱 개발·운영 | 앱마켓 앱의 대부분 영역을 UI부터 Data까지 기능 단위로 개발·운영했고, 스티커스토어·미니게임·웹게임 같은 새 서비스를 연동했습니다. 제주항공 iOS 앱을 담당했습니다. |

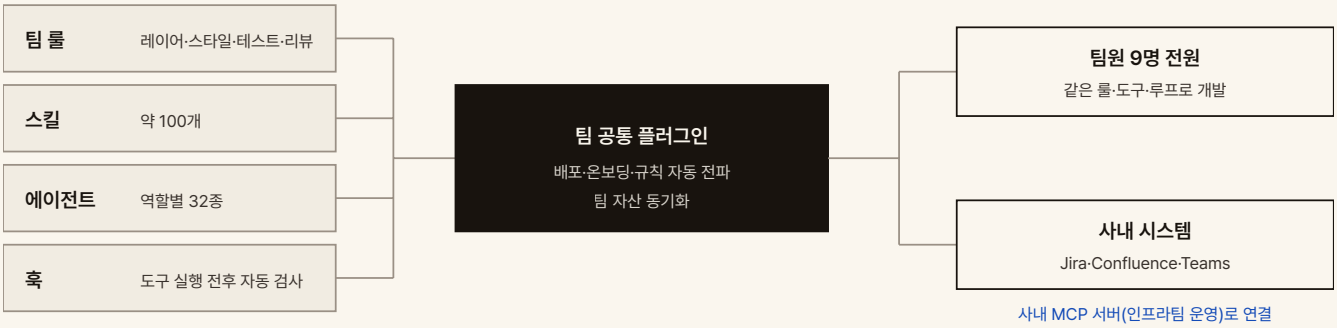
### 차례

팀 AI 하네스: 구축과 다섯 가지 목표 2026.01~현재 2
팀 AI 하네스: 루프와 최적화 2026 3
팀 AI 하네스: 에이전트 설계와 규칙 2026 4
팀 AI 하네스: 품질 자동화 2026 5
팀 AI 하네스: 업무 자동화·지식 2026 6
팀 AI 하네스: 실무 적용 2026 7
앱 아키텍처 전환과 리팩토링 2022.07~2025.09 8
팀 기술 리딩·역량 개발 2022.09~현재 9
다운로드·설치와 로그인 2020.05~현재 10
신규 서비스 연동: 광고·웹게임·미니게임 2025~2026 11
AAB Dynamic Delivery SDK 2020.10~2021.10 12
이전 경력, 학력, 기술 2010~2020 13

# 팀 규칙과 도구, 검증 장치를 묶어 팀 전원이 쓰는 AI 개발 환경을 만들었습니다

하네스는 코딩 에이전트에 팀 규칙·도구·검증 장치를 묶은 개발 환경입니다. 개발 방식이 AI 중심으로 바뀌는 흐름을 받아들이기로 하고, 설계·구축해 팀 전원에 적용했습니다. 아래 다섯 가지 목표에 맞춰 구성 요소를 나눴고, 지금은 클라이언트 팀원 전원이 이 환경으로 개발합니다.

| | |
|---|---|
| 기간 | 2026.01~현재, 단계적으로 고도화 |
| 대상 | 원스토어 Android 클라이언트 팀(9명)의 공통 Claude Code 플러그인과 앱 저장소의 AI 규칙 |
| 역할 | 설계·구축·운영 |




적은 토큰으로 필요한 것만

## 컨텍스트

세션마다 항상 읽는 룰은 진입 파일 하나로 최소화하고, 레이어 룰은 코드 파일을 다룰 때만 붙입니다. 팀 메모리는 색인만 상시로 두고 본문은 필요할 때 읽습니다. 요청이 들어오면 훅이 스킬 색인과 대조해 맞는 스킬을 추천합니다.

정해진 절차대로 고치고 관리

## 지시·도구

팀 룰(레이어별 아키텍처, 코딩 스타일, 테스트, 리뷰 기준)과 에이전트 매뉴얼, 약 100개 스킬이 절차를 정합니다. Cursor에서 쓰던 룰·스킬 체계를 Claude Code로 옮기면서 외부 스킬·에이전트는 팀 사정에 맞게 고쳐 들였습니다. Jira·Confluence·Bitbucket·Teams는 사내 MCP 서버(인프라팀 운영)로, 지식 검색과 판정은 CLI와 스크립트로 처리하고, MCP 사용 규칙은 훅이 주입하며 호출 실패는 기록합니다.

잘못 동작하지 않게

## 통제

6개 이벤트에 훅 25개를 걸어 실행 직전에 막습니다. 되돌릴 수 없는 명령, 보호 브랜치 push와 원격 브랜치 삭제, 외부 전송과 자격 증명 우회, master·develop 대상 PR, 무인 실행 중 질문, 검사 기준 파일 편집이 대상입니다. 검증 에이전트는 결과 확인용 명령만 쓸 수 있습니다. 밖에서 읽어 온 텍스트는 데이터로만 다루게 팀 룰에 넣고, 정상 작업을 막던 가드의 오탐은 찾아 풀었습니다.

착각하지 않게

## 컨텍스트 품질

커밋할 때는 커밋 규약을, PR·위키 도구를 쓸 때는 리뷰·문서 규칙을, 코드 파일을 열 때는 그 레이어의 규칙을 그 시점에 넣습니다. 에이전트 스키마로 에이전트마다 상태와 특성을 정의하고, 에이전트는 역할마다 도구를 제한하며, 팀 메모리에는 마스킹·일반화·승인을 거친 항목만 올립니다.

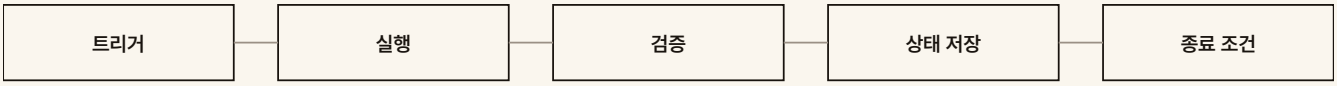
스스로 확인하고 고치게

## 검증·피드백

push 직전에 저장소별 게이트(외부 노출 컴포넌트 스냅숏, 리뷰 기준 동기화 등)를 돌려 실패하면 막고 사유를 돌려줍니다. 수정한 코드는 ktlint로 검사하고, 빌드·테스트가 실패하면 원인을 고쳐 최대 3번 다시 시도합니다. 훅과 스크립트 자체도 테스트로 검증합니다.

# 그 위에 멈출 줄 아는 루프를 올리고 하네스를 가볍게 다듬었습니다

루프는 트리거, 실행, 검증, 상태 저장, 종료 조건으로 설계해 개발·리포트·보고 흐름에 넣었습니다. 최적화는 컨텍스트를 줄이고, 규칙으로 정할 수 있는 일은 LLM 밖으로 옮기는 방향으로 했습니다.




목표 달성 · 회차·시간 상한 · 정체 · 기준 변경 시 멈추고 사람에게

| | |
|---|---|
| 트리거 | **사람이 시작을 정합니다**<br>한 줄 명령으로 무인 루프를 시작하거나, 영업일 테스트 리포트·매주 주간보고처럼 정해진 주기로 돕니다. 시작 전에 작업 설명, 브랜치, 깨끗한 작업 트리, 알림 대상을 확인하고 하나라도 빠지면 시작하지 않습니다. |
| 실행·검증 | **테스트부터, 완료는 검증으로**<br>분류, 계획, 실패 테스트, 구현, 리뷰, 검증, 기억, 개선 순으로 돕니다. 완료는 모델의 말이 아니라 검증 결과가 통과했을 때만 인정합니다. |
| 상태·종료 | **기록을 남기고, 멈출 때를 정해 둡니다**<br>회차마다 사유와 결과를 기록하고 상태 파일을 남겨 이어서 돌 수 있습니다. 회차 수와 세션·회차 시간에 상한을 두고, 같은 반려 사유가 3번 이어지면 정체로, 검사 기준 파일이 바뀌면 무효로 멈춥니다. 어떤 경로로 끝나든 Teams 카드로 사람에게 다음 할 일을 넘깁니다. |

## 하네스 최적화

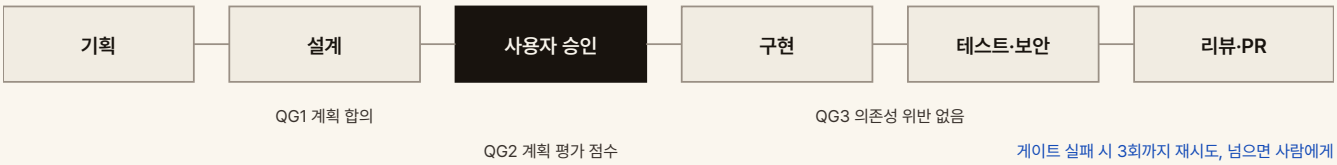
**컨텍스트 경량화:** 상시 룰을 진입 파일 하나로 묶고, 레이어 룰은 조건부로, 레거시처럼 경로 규칙에 안 걸리는 파일은 훅이 모듈과 내용으로 레이어를 판정해 한 번만 주입합니다. 문서 압축으로 토큰을 줄이고, 세션 시작 컨텍스트를 항목별로 산정합니다. **결정론화:** 커밋·PR 규약, push 전 게이트, 아키텍처 판정처럼 규칙으로 정할 수 있는 검사는 훅과 스크립트로 옮겨 LLM 밖에서 돌립니다. **지식 외부화:** LLM Wiki와 팀 메모리는 검색 CLI와 점검 도구로 다룹니다.

**측정:** 세션 기록에서 스킬 호출, 한 번도 안 쓰인 스킬, 토큰, 훅 발동을 집계하는 도구를 만들어 팀원 사용 데이터를 모았고, 검증 판정에 쓸 모델은 비교 실측해 정했습니다. 리뷰 강도를 단계로 나눠, PR 한 건 실측에서 토큰을 3.70M에서 1.49M로 줄이면서 핵심 결함 2건은 그대로 잡았습니다.

# 역할을 나눈 에이전트에 규칙과 검증 장치를 함께 넣었습니다

하네스 안의 에이전트를 직접 설계했습니다. 기능 개발을 역할별로 나눠 맡는 팀, 큰 작업을 계층으로 나눠 맡는 편제, 결과를 작업자와 따로 판정하는 검증 에이전트를 만들고, 각 에이전트가 지킬 규칙을 정의와 훅에 담았습니다.

| | |
|---|---|
| 규모 | 에이전트 32종 |
| 대상 | 원스토어 Android 클라이언트 팀 공통 Claude Code 플러그인 |
| 역할 | 구조·규칙 설계, 작성, 운영 |




기능 개발 팀

### 역할별 에이전트 7개와 세 게이트

오케스트레이터가 기획·설계·UX·구현·테스트·보안·리뷰 에이전트를 단계별로 부릅니다. 전체·빠른 수정·보안 중심 세 모드가 있고, 에이전트끼리는 직접 부르지 않고 파일 산출물로 주고받습니다. 규칙은 프로젝트 › 오케스트레이터 › 워크플로 › 개별 에이전트 순으로 우선합니다.

3계층 편제

### 큰 작업을 나누되 사람이 처음에 승인

사람이 편제 계획과 완수 기준(DoD), 토큰 예산을 착수 전에 한 번 승인하면, 총괄이 구역을 나눠 분대장 에이전트를 부르고 분대장은 작업 성격에 따라 하위 모델을 배정합니다. 빌드·git·공용 파일은 총괄만 직렬로 다루고, 분대장은 명세만 보고 임무를 재진술해 의도를 맞춘 뒤 착수합니다. 예산을 넘으면 새 소환을 멈춥니다.

검증 에이전트

### 작업자와 판정자를 분리

1차·최종 검증과 완수 검증을 작업자와 다른 읽기 전용 에이전트가 맡고, 보고 글이 아니라 결과 파일을 다시 확인해 판정합니다.

업무 보조

### 개발 밖의 일도 에이전트로

Jira·Bitbucket·Teams·Outlook을 모아 할 일 순서를 추천하는 업무 비서, 개인 메모리를 마스킹해 승인 후 팀 자산으로 올리는 큐레이터, 요건 분석 보고서, PR 리뷰 반영, SonarQube 수정 에이전트를 만들었습니다.

## 설계 원칙

에이전트는 역할별로 특화하고, 서로 직접 부르지 않고 파일 산출물로 주고받으며, 사람이 승인하는 지점을 정해 둡니다.

# 테스트, 리뷰, 크래시, 정적 분석을 하네스가 맡아 돌게 했습니다

사람이 챙겨야 돌던 품질 작업을 스킬과 에이전트로 옮겼습니다. 테스트는 정해진 주기에 사람 없이 돌고, 리뷰·크래시·정적 분석 이슈는 AI가 먼저 분석하고 고친 뒤 사람이 승인합니다.

| | |
|---|---|
| 기간 | 2026 |
| 대상 | 원스토어 국내·글로벌 Android 앱의 테스트·리뷰·크래시·정적 분석 |
| 역할 | 워크플로·스킬·에이전트 설계와 구축, 운영 |

| | |
|---|---|
| 테스트 자동화 | **작성부터 정기 실행과 보고까지**<br>계층별 테스트 작성 가이드를 하네스 스킬로 만들어 여러 에이전트가 Kotest(BehaviorSpec)·MockK 테스트를 나눠 쓰게 했고, Domain 계층 UseCase 전체와 Repository 구현체에서 앱 전역으로 단위 테스트를 넓혔습니다. Jenkins 단위 테스트는 사람 없이 도는 무인 실행으로 바꿔, 영업일마다 결과를 위키 회차 페이지로 만들고 Teams 결과 카드(빌드 실패 카드 포함)로 알립니다. 단말이 연결돼 있으면 계측 테스트 축을 더 돌리고, 빌드 판정에는 근거와 집계 범위를 붙입니다. 컴파일되지 않던 계측 테스트도 복구했습니다. 지금은 코드를 쓰면 단위 테스트를 함께 추가하는 것이 팀 규칙이고, 하네스 도입 뒤 QA 검증 단계에서 검출되는 이슈가 줄었습니다(비율은 집계하지 않았습니다). |
| 실기기 UI E2E | **화면, 로그, 서버 응답을 한 번에 검증**<br>Maestro와 mobile-mcp로 UI E2E 테스트 러너를 만들었습니다. 시나리오를 쓰고 실기기에서 돌리며, 화면과 logcat, 서버 전문 세 축을 함께 확인합니다. |
| 코드 리뷰 | **데일리 리뷰와 통합 리뷰**<br>데일리 코드 리뷰는 하루 변경분을 분석하고 지난 이슈의 이월을 추적해 Confluence KPI 대시보드와 Teams 알림으로 남깁니다. 통합 리뷰는 검사 항목을 결정적·LLM·위임 검사로 나누고 단계별 수준으로 아키텍처·코드 결함을 판정해 Bitbucket에 한글 댓글을 답니다. 받은 리뷰 댓글은 수정 필요와 개선으로 나눠 승인 후 반영합니다. |
| 크래시·정적 분석 | **분석과 수정까지, 반영은 사람이 승인**<br>Firebase Crashlytics 링크나 스택트레이스를 주면 원인을 찾고, 승인 후 수정과 PR까지 이어집니다. SonarQube는 이슈를 모아 등급 순으로 최소 수정하고 빌드로 검증한 뒤 규칙별로 커밋하며, 동작이 바뀔 수 있는 이슈는 건너뜁니다. LeakCanary를 켠 빌드에서 adb·monkey로 런타임 메모리 누수를 찾는 스킬도 만들었습니다. |

자동화로 고친 크래시·이슈 건수는 따로 집계하지 않았습니다.

# PR부터 디자인, 업무, 지식까지 스킬과 워크플로로 묶었습니다

한 가지 일을 하는 단일 스킬을 만들고, 그것을 이어 붙인 워크플로로 개발과 업무를 자동화했습니다. 품질 자동화를 뺀 나머지 가운데 의미 있는 것만 골랐습니다.

| | |
|---|---|
| 규모 | 팀 플러그인 스킬 약 100개 |
| 대상 | 팀 공통 Claude Code 플러그인, 팀원별 로컬 LLM Wiki |
| 역할 | 설계·작성, 팀 배포·개선 |

| | |
|---|---|
| PR 흐름 | **PR 작성부터 리뷰 반영까지 한 흐름으로**<br>브랜치의 커밋 범위를 분석해 Jira 티켓·작업 유형·레이어를 담은 한글 PR 메시지를 만들고, Bitbucket PR을 올린 뒤 통합 코드 리뷰로 한글 댓글을 답니다. 받은 댓글은 필수 수정과 개선으로 나눠 승인한 항목만 고칩니다. |
| 구조 정리 | **정리 대상은 한 목록으로, 외부 계약은 스냅숏으로**<br>미사용 코드, deprecated API, minSdk 기준으로 필요 없어진 코드, 의존성을 하나의 목록으로 냅니다. Java→Kotlin 전환 스킬은 테스트를 먼저 요구하고 회귀 게이트를 통과해야 넘어갑니다. 다른 앱과 맺은 AIDL·컴포넌트·권한 계약은 스냅숏으로 떠 두고 변경 전후를 비교합니다. |
| Figma 연동 | **디자인을 읽어 UI 개발**<br>Figma 디자인 덤프·검색 스킬로 화면 구성과 값을 가져와 UI 개발에 썼습니다. |
| 업무 자동화 | **반복 업무를 스킬로**<br>Teams 멘션 수집과 자동응답, Jira 필터 감시 카드, PR 리뷰 요청·완료 알림, Jira 하위 일감 생성, 주간·일일 보고와 Confluence 발행(주간보고는 매주 무인 작성·게시), 번역 시트에서 strings.xml 생성, 기술 트렌드·구글 정책 보고서를 스킬로 만들었습니다. |
| 로컬 LLM Wiki | **산출물을 관리하고 환각을 억제하는 지식베이스**<br>공개된 LLM Wiki 개념을 참고해 팀원별 로컬 지식베이스를 만들었습니다. 형식과 값 범위를 정한 마크다운 문서에 산출물을 3단계로 쌓고, 잘못된 위치에 저장하면 자동으로 막습니다. 검색 CLI·뷰어·점검 도구를 팀 도구로 옮기고, 업무 도메인과 키워드를 매핑해 어떤 에이전트든 과거 산출물과 도메인 정보를 빠르게 찾게 했습니다. |
| 메모리 편입 | **개인 메모리를 팀 하네스로**<br>개인 메모리에 쌓인 교훈과 방식 중 팀에 쓸 만한 것을 룰·스킬·훅으로 만들어 넣었습니다. 큐레이터가 시크릿과 식별자를 가리고 일반화한 뒤, 사용자가 승인해야 팀 자산이 됩니다. |

# 만든 하네스로
# 실제 앱 구조를 고치고 새 영역을 열었습니다

앱 전체 진단을 하네스의 스킬과 매뉴얼로 만들어 자동화하고, 그 결과로 금지 의존을 걷어 냈습니다. 같은 하네스로 처음 다루는 사업 영역도 빠르게 개발했습니다.

| | |
|---|---|
| 기간 | 2026 |
| 대상 | 원스토어 국내 Android 앱 전체 모듈, 신규 서비스 개발 |
| 역할 | 진단 자동화, 구조 개선, 팀 교육과 회사 공유 |

| | |
|---|---|
| 1 도구 | **진단을 스킬과 매뉴얼로**<br>모듈과 레이어 의존이 규칙을 어기는지 판정하고 리포트와 의존성 그래프를 내는 작업을 하네스의 스킬과 매뉴얼로 만들어, 개선 대상 목록이 자동으로 나오게 했습니다. |
| 2 진단 | **앱 전체 검증과 레거시 인벤토리**<br>신규 광고 연동 모듈에 먼저 써 본 뒤, 국내 앱 전체를 판정하고 레거시 개선 대상 목록을 만들었습니다. |
| 3 정리 | **금지 의존을 코드에서 제거**<br>공용 Data 모듈이 갖고 있던 구현 모듈 연결을 각 모듈로 옮기고, 데이터 소스 구현을 모듈 안으로 감췄습니다. 계층 경계를 어긴 참조를 없애고 최소 지원 버전 아래의 코드·리소스를 정리했습니다. |
| 4 재발 방지 | **사람과 AI에 같은 기준**<br>레이어 의존 위반이 늘면 잡는 검사를 붙이고, 같은 규칙을 AI가 코드를 고칠 때도 주입되게 했습니다. |

## 처음 다루는 영역을 빠르게

웹게임과 광고 플랫폼처럼 기존과 다른 사업 영역을 하네스로 빠르게 도입했습니다. 팀은 웹게임 플랫폼 앱을 광고 연동까지 약 3주에 개발했고, 도입 뒤 QA 검증 단계에서 검출되는 이슈가 줄었습니다.

## 팀 교육과 회사 공유

팀 학습용으로 Claude Code 에이전트 가이드 12편(수명주기, 훅, 워크플로 오케스트레이션, MCP, 루프 엔지니어링, 온톨로지)과 기초 가이드 2편을 사내 위키에 썼습니다. 하네스 구축 경험은 회사 대상 AI 발표와 부서 간 공유로 전했습니다.

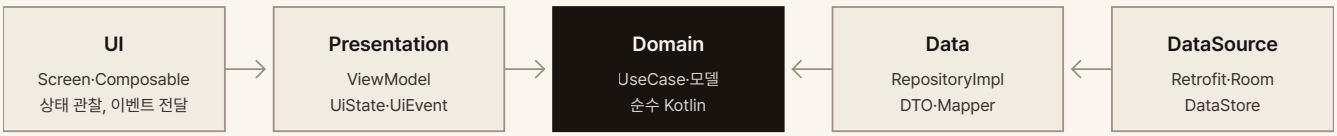
## 기술

Kotlin, Gradle 멀티 모듈, 의존성 그래프, Claude Code 워크플로

# 오래된 상용 앱의 구조를 서비스를 멈추지 않고 바꿨습니다

원스토어 앱은 15년 된 프로젝트라 MVC·일부 MVP 코드와 기술 부채가 쌓여 있었습니다. 샘플 아키텍처로 방향을 팀에 설명한 뒤, 기능 개발과 병행하며 단계별로 범위를 넓혀 국내·글로벌 두 앱을 새 구조로 옮겼습니다.

| | |
|---|---|
| 기간 | 2022.07~2025.09, 기능 개발과 병행한 단계별 진행 |
| 대상 | 원스토어 국내·글로벌 Android 앱 |
| 역할 | 2022.07부터 아키텍처 개선 주도, 2023년부터 기술 리드. 설계와 직접 리팩토링, 기술 과제 주도 |




의존은 Domain을 향하고, 의존성 역전은 Repository와 DataSource 두 경계에 둡니다

| | |
|---|---|
| 2021.09~12 | **첫 적용: 스티커스토어**<br>스티커팩·이모지 구매·편집 앱을 RxJava, MVVM + 단방향 데이터 흐름, Retrofit, 수동 DI 구조로 개발했습니다. 새 구조를 처음 적용한 앱입니다. |
| 2022.07~2023.04 | **설계와 시범 적용**<br>새 구조를 샘플 프로젝트로 만들어 팀에 설명했습니다. 1차 모듈 분리에서 domain 모듈의 Android 의존을 인터페이스로 걷어 냈고, 2023년 3~4월 계층별 모듈 구조를 채택해 스켈레톤을 만들었습니다. |
| 2023.04~06 | **글로벌 앱 1단계: 레이어 분리**<br>이관받은 글로벌 앱을 새 구조로 다시 잡았습니다. 공통 UI 컴포넌트와 스토어 화면을 별도 모듈로 나눴습니다. |
| 2023.12~2024.03 | **글로벌 앱 2단계: 설치 모듈 분리, 멀티 브랜드 빌드**<br>설치 기능을 설치 모듈로 떼어 여러 설치 경로를 하나의 구조로 묶고, 브랜드별 모듈로 멀티 브랜드 빌드를 만들었습니다. |
| 2024.01~11 | **국내 앱 데이터·UI 계층**<br>데이터 계층(2024.01~07)은 Hilt를 들여와 데이터 매니저 싱글턴을 없애고, 콜백 기반 데이터 로더를 Flow 기반 UseCase로 바꿨습니다. 관련 코드를 Kotlin으로 옮기고, 화면별 Manager를 하나씩 Repository·UseCase로 이관했습니다. 이어서 UI 계층(2024.08~11)을 리팩토링했습니다. |
| 2025.06~09 | **Service → WorkManager 전환, 다운로드·설치 기능 리팩토링**<br>팀 과제인 Service → WorkManager 전환에서 다운로드 서비스가 맡던 경로를 Worker와 UseCase로 옮기고 서비스를 없앴습니다. 다운로드·설치 기능 리팩토링도 함께 진행했습니다. |

## 화면 계층

레거시 구조(MVC·일부 MVP)를 MVVM + 단방향 데이터 흐름(Event-State 패턴)과 MVI로 바꿨습니다. Hilt로 계층 사이를 추상화하고 비동기는 Coroutine·Flow로 옮겼습니다. Compose 전환은 팀에 전파하고 상세·마이페이지 등 화면 전환에 참여했습니다.

## 기술

Kotlin, Clean Architecture, MVVM·MVI, Hilt, Coroutine·Flow, Room, WorkManager, Jetpack Compose, 멀티 모듈

# 바뀐 방식이 팀의 방식이 되도록 가르치고, 규칙으로 만들고, 검사로 지켰습니다

구조를 바꾸는 것만큼 팀이 그 구조를 따르게 하는 일이 중요했습니다. 2023년부터 원스토어 클라이언트 팀(9명)의 기술 리드로 아키텍처 표준과 코드 리뷰 문화를 세우고, 기반 기술 도입을 결정하고, 기술 전환 과제를 팀 과제로 이끌었습니다.

| | |
|---|---|
| 기간 | 2022.09~현재 |
| 대상 | 원스토어 Android 클라이언트 팀(9명) |
| 역할 | 기술 리드(공식 직함 매니저): 아키텍처 표준, 리뷰 문화, 기반 기술 도입, 기술 과제 주도 |

**가르치기**
아키텍처·비동기·테스트 주제로 팀 학습을 꾸준히 진행, 본부·전사 발표

**규칙으로**
코드 리뷰 가이드를 팀 규칙에 반영, 계층별 테스트 작성 가이드

**검사로 지키기**
팀 룰로 만든 기준을 하네스의 게이트와 판정 검사로 옮겨 사람과 AI가 같은 기준을 따르게

약속으로만 두지 않고, 개발 과정에서 기준이 지켜지게 만드는 순서입니다.

## 기술 방향과 과제 주도

- 새 아키텍처를 설계하고 계층별 모듈 구조와 개발 표준을 팀 합의로 확정했습니다. Coroutine·Flow·Hilt·WorkManager 등 기반 기술의 도입을 결정하고, Compose 전환을 팀에 전파했습니다.
- Service → WorkManager 전환과 2026 기술 부채 개선 과제를 팀 과제로 이끌었습니다. App Startup, 레거시 리팩토링 전환도 팀 과제로 진행했습니다.

## 리뷰와 병합

리뷰어로 팀원 PR을 검토하고, 팀원 작업분을 포함한 대규모 리팩토링 병합을 맡았습니다(2025.01).

## 팀 학습에서 다룬 주제

| | |
|---|---|
| 설계 | MVVM, MVI, 단방향 데이터 흐름, UiState·UiEvent, Clean·헥사고날 아키텍처, 멀티 모듈, SOLID·DRY, DDD |
| 비동기·DI | Coroutine, Flow, RxJava, Hilt, Koin |
| UI·Jetpack | 명령형·선언형 UI 비교, Compose Navigation, ViewModel, App Startup, WorkManager, Kotlin |
| 테스트 | TDD, 테스트 코드 작성법, MockK, UI 테스트, 코드 리뷰 방법 |
| 앱 구조 | APK·AAB 구조, 서명 키 로테이션, Dynamic Delivery |
| AI | Cursor, Claude Code, 에이전트·훅·MCP, 루프 엔지니어링 |

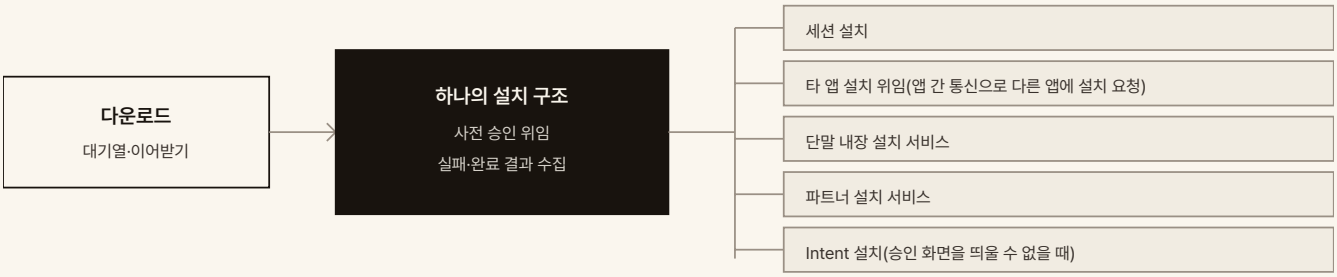
## 발표

하네스 구축 경험을 회사 대상으로 발표했고, 본부 공유도 여러 차례 했습니다.

# 앱마켓에서 설치가 실패하면 사용자는 앱을 받지 못합니다

2020년 5월부터 다운로드와 설치를 맡아 왔고, 로그인·인증도 담당했습니다. 앱 종료, 망 전환, 단말 차이처럼 정상 흐름 밖의 경로를 하나씩 막았습니다.

| | |
|---|---|
| 기간 | 다운로드·설치 2020.05~현재, 로그인·인증 2020~2026 |
| 대상 | 원스토어 국내·글로벌 앱의 다운로드, 설치, 자동 업데이트, 로그인 |
| 역할 | 다운로드·설치와 자동 업데이트, 로그인·인증 담당(개발·운영) |




| | |
|---|---|
| 다운로드 | **끊겨도 이어지게**<br>Wi-Fi·데이터 전환 때 이어받기, 앱이 종료돼도 설치까지 복구하는 흐름, 일시 중지 항목 일괄 재개 때 순서가 뒤집히던 오류 수정, 알림 채널·권한 상태에 따른 알림 처리를 맡았습니다. |
| 설치 | **단말이 달라도 설치되게**<br>단말이 사전 승인을 막으면 설치 시점 승인으로 넘기고, 설치 중 보안 예외가 나면 다음 설치부터 다른 방식으로 바꾸는 대체 경로를 넣었습니다. 설치가 안 되는 단말은 서버와 동기화한 로컬 DB로 관리해 설치 방식을 바꿨습니다(국내·글로벌). |
| 자동 업데이트 | **서버 호출을 줄이게**<br>글로벌 앱의 다운로드 인증 토큰 조회를 필요할 때만 하게 하고, 오래 로그인하지 않은 사용자의 백그라운드 자동 업데이트 작업은 멈추게 했습니다. |
| 로그인·인증 | **로그인이 흐름을 끊지 않게**<br>성인·실명 인증과 로그인 잠금, 외부 앱·SDK에서 진입할 때의 로그인, 토큰 만료 시 자동 연장을 만들었습니다. |

## 홈 화면 성능

홈의 중복 데이터 호출과 선택하지 않은 패널의 사전 로딩을 줄이고 상품 데이터 조회를 비동기로 바꿔 홈 동작 시간을 줄였습니다.

## 기술

Kotlin, Java, PackageInstaller, Foreground Service, WorkManager, Coroutine, Room, Retrofit, Firebase, 토큰·세션 관리

# 새 서비스가 들어올 때마다 앱이 받을 수 있는 구조부터 만들었습니다

미니게임, 웹게임, 광고 플랫폼처럼 스토어에 새로 들어온 서비스를 앱 쪽에서 연동했습니다. 한 서비스용으로 만든 광고 연동은 국내·글로벌 앱이 함께 쓰는 공용 모듈로 키웠습니다.

| | |
|---|---|
| 기간 | 미니게임 2025.11~2026.01, 광고·웹게임 2026.07~10 |
| 대상 | 원스토어 국내·글로벌 앱의 미니게임, 웹게임(H5), 보상형 광고 |
| 역할 | 앱 연동 개발, 공용 광고 모듈 설계·구현, 신규 광고 서버 연동 |




2026.07~10

### 광고 연동을 공용 모듈로

외부 게임 서비스용 광고 JavaScript 브리지를 만들고, 한 서비스 전용이던 광고 실행 계층을 웹게임과 함께 쓰게 나누고, 공용 모듈을 별도 저장소로 떼어 국내·글로벌 앱에 붙였습니다. 서명 기반의 새 광고 서버 규격으로 지면 조회를 바꾸고 광고 토큰 등록을 추가했습니다.

2026.08~09

### 웹게임 플랫폼 앱

H5 웹게임을 원스토어 앱 안에서 실행하는 플랫폼을 광고 연동까지 포함해 개발했습니다. 웹게임의 마이크·카메라 권한을 앱이 대신 처리하게 하고, 개발사가 테스트 빌드를 실행하는 경로를 만들었습니다.

2025.11~2026.01

### 미니게임 서비스

별도 프로세스로 도는 미니게임이 로그인 정보와 연령 확인을 얻도록 AIDL·ContentProvider 인터페이스를 만들고, 홈 화면 바로가기와 광고 SDK 의존성 충돌을 처리했습니다.

## 어려웠던 지점

웹게임 광고가 지면 조회와 토큰 등록에서 실패했습니다. 웹게임 상품 식별자가 서버가 기대한 자리와 다른 곳으로 나가고 있었습니다. 서비스마다 흩어져 있던 식별자 선택을 한 곳으로 모아 고쳤습니다.

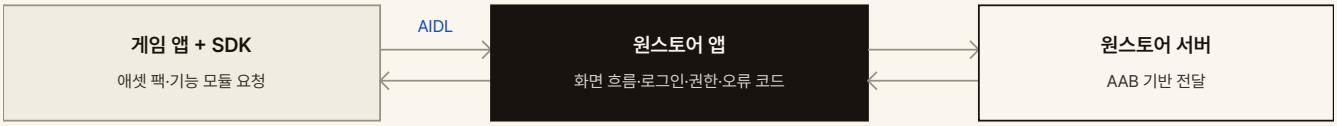
## 기술

Kotlin, WebView, JavaScript 브릿지, 광고 SDK(Verse8·넵튠 ADX), 멀티 모듈, REST API 연동, AIDL, ContentProvider

# 게임의 대용량 리소스를 원스토어에서도 필요할 때 받게 했습니다

게임사는 Google Play의 동적 전달(Dynamic Delivery)로 기능 모듈과 애셋 팩을 필요할 때 내려받게 합니다. 원스토어에서도 같은 방식이 되도록 Play Core의 동작을 분석해 SDK를 설계하고, 원스토어 앱과의 연동을 개발했습니다.

| | |
|---|---|
| 기간 | 2020.10~2021.10 |
| 대상 | 게임·앱 개발사용 AAB Dynamic Delivery SDK와 원스토어 앱 연동 |
| 역할 | 기술 분석·설계, SDK·앱 연동 개발 |




| | |
|---|---|
| 1 분석 | **Play Core와 AAB 구조**<br>Google Play Core의 기능 모듈·애셋 전달 동작과 AAB 구조를 파악해 원스토어에서 같은 방식이 되기 위한 요건을 정리했습니다. |
| 2 설계 | **인터페이스와 연동 규격**<br>SDK 기본 인터페이스와 필요 기능을 정의하고, 서버 연동 규격을 정했습니다. |
| 3 구현 | **SDK와 원스토어 앱 연동**<br>SDK와 앱 사이의 AIDL 인터페이스와 전달 데이터 규격을 설계·구현하고, 애셋 팩 상태·요청·취소·삭제를 만들었습니다. SDK에서 앱으로 넘어올 때의 화면 흐름, 로그인, 권한, 강제 업데이트 확인과 오류 코드를 구현했습니다. |
| 4 안정화 | **앱 종료와 망 전환**<br>다운로드 session ID를 저장해 앱 종료 뒤 나던 다운로드 오류를 해결하고, Wi-Fi 연결이나 셀룰러 데이터 허용으로 바뀌면 다운로드를 재개하게 했습니다. 라이브러리 내부 Lock으로 생긴 ANR도 고쳤습니다. |

## 개발사 지원과 결과

검증용 샘플 앱과 SDK 개발 가이드를 썼고, AAB 분할 전달의 전제가 되는 단말 사양 수집·등록 기능도 만들었습니다. SDK는 원스토어 앱에 탑재되는 형태로 일부 기능을 상용 적용했습니다.

## 기술

Android App Bundle, Play Core(AssetPackManager·SplitCompat), AIDL, bundletool, protobuf, ProGuard, Android 라이브러리 배포

# 이전 경력, 학력, 기술

## AKIS, 항공서비스팀 과장

2018.07~2020.03

| | |
|---|---|
| 2018.07~2020.03 | **제주항공 고객용 앱과 항공사 사내 앱.** iOS(Swift) 앱을 담당해 개발·운영했습니다. 항공 예약 파트(3명)를 리드하며 웹 개발을 함께 했고, 예약 웹 화면과 연동되는 iOS 네이티브 브릿지를 구현해 App Store에 업데이트를 배포했습니다. 항공 스케줄·정비 등 사내 앱은 유지보수·고도화하고 iOS 엔터프라이즈 인증서로 사내 배포했습니다. |

## 미래ICT, 프리랜서

2018.04~2018.06

| | |
|---|---|
| 2018.04~2018.06 | **현대모비스 차세대 내비게이터 선행 개발 참여.** 차량 마이컴(MCU) 연동 HIDL HAL 포팅(AOSP, C++) |

## 이노에이스(현 에스피테크놀러지), 모바일서비스 개발3팀 과장

2010.01~2017.10

| | |
|---|---|
| 2014.07~2017.10 | **앱마켓 앱: 티스토어(원스토어 전신)·원스토어.** UI 레이어와 사용자 비즈니스 로직 담당. 전시·상세·검색·선물·멀티구매 화면과 웹툰 상품 화면, 원스토어 6.0 UI 전면 개편 |
| 2015.01~2017.10 | **선탑재 보조 앱 Utility·Setup Wizard.** 신규 화면 개발과 고객 문제 대응 |
| 2012.08~2014.03 | **T간편모드(SKT 기본 탑재 런처).** 2G폰 키패드형 홈 UI, 시스템 권한으로 단말 설정 제어, 제조사 단말 포팅 |
| 2011.07~2012.07 | **Rabbit Launcher·SmaDeco.** 3D 런처의 엔진 렌더링 연동, 테마 상점, 인앱 결제·포인트 서버 연동, 일본 현지화 |
| 2010.08~2011.12 | **폰꾸미기(SKT 선탑재, 400만 이상 다운로드).** 3D UI 엔진 렌더링 연동, 테마 상점·홈 UI, 제휴사용 테마 제작 도구, 출시 후 운영 |
| 2010~2014 | **그래픽 미들웨어·SDK.** 3D UI 프레임워크(MUIF) 엔진 디버깅·컴포넌트 제작, Android View와 Box2D 물리엔진을 결합한 SDK, AR 브라우저의 동영상 재생 기능 |

## 학력, 병역, 자격

| | |
|---|---|
| 학력 | 단국대학교 컴퓨터과학과 학사(경영학 복수전공) (2003.02~2010.02) |
| 병역 | 공군 병장 만기 제대 (2004.10~2007.01, 전산병) |
| 자격 | 정보처리기사 (2009) |

## 기술

| | |
|---|---|
| 언어 | Kotlin, Java, Swift, C++, C, Lua |
| Android | Jetpack(Lifecycle·ViewModel·Room·WorkManager·Paging·App Startup·Compose), Coroutine·Flow, RxJava, Hilt, Retrofit·OkHttp, RecyclerView, PackageInstaller, AAB, AIDL, WebView |
| 품질·배포 | Kotest·MockK 단위 테스트, Android 계측 테스트, Maestro 실기기 UI 테스트, Firebase Crashlytics, Jenkins, 아키텍처 의존성 검증, Google Play·App Store 배포 |
| AI 개발도구 | 팀 AI 하네스 구축(룰·스킬·에이전트·훅·개발 루프·로컬 LLM Wiki), Claude Code(플러그인·스킬·훅·에이전트), MCP, Cursor |
| 협업 | Git, Bitbucket, Jira, Confluence, Teams |